

VetAI — The Security Layers

Every guard around the agentic AI, what it stops, and where it lives
in the code.

Read from the running source, not from design notes

backend/src/ai/ · 2025 / 2026

The one idea behind all of it

A language model is treated here as a **useful but untrusted component**. It is never the thing that decides whether something is safe, who you are, or what gets written to the database. Every guard below exists to keep a decision away from the model — either by making it before the model runs, or by checking the model's work afterwards. Where a rule could be enforced by code instead of by a prompt, it is.

Input guardrails — before the model

backend/src/ai/safety.js

Three detectors run on the raw message. If any fires, a fixed answer is returned and **the model is never called at all**.

Emergency detection

`detectEmergency()` · `emergencyResponse()`

In plain terms: if someone says their dog ate chocolate or their cat can't urinate, they get told to go to a vet *now* — instantly, and without asking an AI first.

Bilingual regular expressions over the message. Matching is deliberately precise rather than broad: a poisoning pattern requires an ingestion verb *near* the toxin, so "what human foods are toxic to dogs?" is a question, not an emergency.

Why not let the model judge it? Because this is the one decision that must never fail. A regex answers in the same few milliseconds every time. A model can be slow, can be rate-limited, can be argued out of a position. We did not want to discover which.

Urgent-symptom detection

`detectUrgent()` · `urgentResponse()`

A second, softer tier — repeated vomiting, diarrhoea, limping, straining. Not an emergency, but it returns 24-hour advice rather than general chat, and it also short-circuits the model.

Toxic-medication refusal

`detectToxicMedication()` · `toxicMedResponse()`

In plain terms: "how much ibuprofen can I give my dog?" gets a refusal, not a number.

Checked *before* the booking flow, so the refusal cannot be skipped by wrapping the question inside a booking request.

Arabic parity — including the gender forms

`safety.js` · `eval.js` `gender-pair tests`

Every detector above is written for Arabic as well as English. Arabic verbs are gendered, so a pattern written only in the masculine form silently fails on the feminine one — the eval suite tests both forms of each phrase specifically to prove that gap is closed.

A real bug this class of thinking caught: `\d` does not match Eastern Arabic-Indic numerals. A dose written ٥٠٠ would have passed a dose filter built on `\d`. The character class used is `[0-9٩-٠]`.

What the model is never handed

backend/src/controllers/aiChatController.js · backend/src/ai/tools.js

Write-capable tools are withheld

aiChatController.js – the READ_TOOLS allow-list

In plain terms: in free-form conversation, the AI physically cannot book, create an account, or register a pet. Those tools are not in the set it is given.

Ten tools exist. The free-form chat path receives only the read-only ones, filtered by an explicit allow-list — not by asking the model to behave:

```
const READ_TOOLS = ['searchMedicalGuidelines', 'findMatingPartners',
                    'findAdoptablePets', 'navigateTo'];
// vet discovery is added only when the vets feature is actually live
if (await isFeatureEnabled('vets')) READ_TOOLS.push('findAvailableVets', 'searchProviders');

const chatTools = Object.fromEntries(
  Object.entries(tools).filter(([k]) => READ_TOOLS.includes(k));
```

Writes happen only inside the deterministic booking state machine, which the model does not orchestrate — it is asked for one narrow thing per turn (pull a name, a date, a pet out of a sentence) and the server decides what to do with it.

Identity is server-owned

tools.js – ctx.userId

Who you are comes from the verified JWT and is injected into the tool context. Nothing the model produces, and nothing in the request body, can change it. Every write authorises against `ctx.userId`.

The account-takeover guard

tools.js – createAccount()

A real vulnerability, found and closed. `createAccount` used to treat an existing email as a successful sign-in and bind the conversation to that account. Because the chat endpoint allows anonymous callers, typing a stranger's registered email was enough to act as them — the next `bookAppointment` would authorise against the victim's id, book on their pet, and email their vet.

The tool now refuses and hands off to login. Proving you own an address is authentication's job, and this tool cannot do it, so it does not try. `ctx.userId` is deliberately left untouched.

Output guardrails — after the model

backend/src/ai/safety.js — screenAssistantReply()

Defence in depth. Even if a model is jailbroken, confused, or simply wrong, its reply is screened before it reaches the user.

What is screened	What happens
A medication dose, in either script	Reply replaced with a refusal, in the user's language
A dangerous home remedy	Same refusal path
The system prompt leaking	Reply replaced with a polite deflection
An empty or blank reply	<code>neverEmpty()</code> guarantees something useful is returned

Why screen your own model's output at all

Because prompts are guidance, not enforcement. A system prompt saying "never give a dose" is an instruction the model usually follows. A regex on the way out is a rule it cannot fail to follow. The first is worth having; only the second is worth relying on.

Anti-hallucination — the route contract

backend/src/ai/appRoutes.js · backend/tests/ai/eval.js

Every link the AI emits is validated against the real router

In plain terms: the assistant cannot send you to a page that doesn't exist.

The AI can return a navigation block that the frontend hands straight to the router. A path that is not a real route falls through to the catch-all and renders a 404 — which is exactly how a non-existent `/pets` once shipped.

Every route the backend may emit is declared once in `appRoutes.js`, and `resolveAppPath()` refuses anything outside it:

```
return { success: false,  
        error: `"$${route}" is not a page on PetPluse. Do not invent links.` };
```

The part that makes it hold: the eval suite parses the real `App.jsx` and asserts the declared list still matches the router. A hallucinated link stops being a content problem and becomes a failing build.

Grounding, and the right to say "I don't know"

backend/src/ai/ragService.js

Health answers are retrieved from a curated knowledge base first — pgvector cosine similarity, with a keyword-ranking fallback, matched on the pet's species. If nothing scores well enough, the service returns nothing rather than the closest partial match.

Deliberate: a "best effort" answer to a question the knowledge base cannot support is the most dangerous kind of answer, and it hides the gap. Returning nothing keeps the gap visible in the logs.

Abuse and cost control

backend/src/ai/chatQueue.js · backend/src/ai/llmClient.js

The concurrency gate

In plain terms: the token budget is shared across the whole account, and serverless instances cannot see each other. So the queue lives in the database, where they can all contend for the same row.

One statement claims a slot, and only if there is room *and* the caller is next in line:

```
WITH bump AS (  
  UPDATE ai_chat_gate  
    SET in_flight = in_flight + 1  
  WHERE id = 1  
    AND in_flight < $1                -- room  
    AND $2 = (SELECT MIN(id) FROM ai_chat_queue) -- your turn  
  RETURNING 1  
)  
INSERT INTO ai_chat_slots (ticket_id)  
SELECT $2 FROM bump;                -- only if the UPDATE matched
```

Two instances cannot both read `in_flight < max` as true, because they contend for a lock on one row rather than independently counting rows in a table.

- **2 concurrent** model calls platform-wide (`AI_CHAT_MAX_CONCURRENT`)
- **FIFO** — a freed slot goes to the oldest waiting ticket, not the luckiest poll
- **45s** maximum wait, then the request degrades to a grounded knowledge-base answer
- **90s** stale-slot reaper, so a crashed instance cannot shrink the gate permanently

Security relevance: without this, a handful of concurrent users — or one determined one — exhausts the shared budget and the assistant stops answering for everybody. Denial of service by cost is still denial of service.

Bounded execution

llmClient.js — `AI_TIMEOUT_MS`

Every model call carries an abort signal with a 90-second ceiling, chosen to sit under the serverless function's own limit so a slow provider degrades gracefully instead of the platform killing the function and surfacing as an outage.

Input bounds and SSRF

aiChatController.js

- Messages are capped at 2,000 characters; history is trimmed to the last 12 turns.
- An optional symptom photo is accepted **only** from our own Cloudinary origin (`^https://res\.cloudinary\.com/`), so a caller cannot make the vision model fetch an arbitrary URL on their behalf.

The security agent is advisory only

backend/src/ai/securityAgent.js

A model explains a block. It never makes one.

A separate AI module triages security events the WAF has already caught. It runs **after** the decision, out of band, and its output is explanatory. Enforcement has already happened by the time it is invoked.

Its answers are then clamped by code that the model cannot override:

```
// SQLi was matched by a regex, not inferred — it is not the model's call.
if (event?.type === 'SQL_INJECTION_ATTEMPT') {
  analysis.riskLevel      = 'CRITICAL';
  analysis.recommendedAction = 'BLOCK';
}
// abuse cannot be talked down below HIGH
if (analysis.riskLevel === 'LOW') analysis.riskLevel = 'HIGH';
// a non-numeric or out-of-range confidence collapses to a safe value
analysis.confidence = Math.max(0, Math.min(1, confidence));
```

The rule, stated once: a SQL-injection signature can never be downgraded below CRITICAL, whatever the model returns. Unparseable output fails to the severe interpretation, not the lenient one.

The endpoint that spends tokens is gated three ways

/api/security/analyze

It consumes model tokens on caller-supplied input, which makes it both an abuse vector and a cost vector. It sits behind authentication, an admin check, and a rate limiter — and the agent forwards only allow-listed fields, so a crafted body cannot smuggle content into the prompt.

Proof — what is actually tested

backend/tests/ai/eval.js · toolContract.test.js · chatQueue.test.js

Suite	Assertions	What it holds
ai/eval.js	141	Guardrails in both languages, Arabic gender pairs, output screening, the route contract against the real router
ai/toolContract.test.js	42	The exact tool set is pinned; every tool has a real description, a Zod schema that rejects bad input, and a callable handler
chatQueue.test.js	12	The ceiling holds under genuine concurrency; a reclaimed slot leaves clean state

The suite runs without a key, and without the network

With `AI_PROVIDER=mock` the whole harness runs in CI making no external model call. That is the point: the safety layer must be verifiable without paying anyone, and it is what lets the guardrails be tested on every push.

Adding a tool now fails the build until the pinned manifest is updated — which is the prompt to add its eval coverage at the same time.

Honest limits

- Model-dependent checks (tool-call accuracy, prompt-injection resistance) self-skip on the mock provider — they need a real model and are run separately.
- The guardrails are pattern-based. They are precise and fast, and they will not catch a phrasing nobody anticipated. They are a floor, not a ceiling.
- VetAI supports decisions. It does not diagnose, and it is not a veterinarian — stated in the product, not only in this document.

The layers, in one table

Layer	Stops	Where
Emergency / urgent / toxic-med detection	An emergency being answered as conversation	ai/safety.js
Arabic parity + Arabic digits	A guard that only works in one language or one script	ai/safety.js
Read-only tool allow-list	The model writing to the database	controllers/aiChatController.js
Server-owned identity	Acting as another user	ai/tools.js
Account-takeover guard	Binding a session to an email you don't own	ai/tools.js
Output screening	A dose, a remedy, or the prompt leaking out	ai/safety.js
Route contract	A confidently invented dead link	ai/appRoutes.js
RAG refusal	A fluent answer with nothing behind it	ai/ragService.js
Concurrency gate	Budget exhaustion, and denial of service by cost	ai/chatQueue.js
Timeout ceiling	A hung provider taking the function with it	ai/llmClient.js
Image origin allow-list	SSRF through the vision path	controllers/aiChatController.js
Severity clamps	A model talking a critical event down	ai/securityAgent.js

If you remember one thing

Every guard here answers the same question — **what happens if the model is wrong?** Not if it is malicious, not if it is attacked, just wrong, which is the normal case. The answer in every layer is that something deterministic already ran, or is about to.